

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Cryptography and Network Security

COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO3: Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.CO (BL4 , BL5)

Assessment Tool Weightage: 40%

QUESTIONS: 5

TOTAL MARKS: 25

EXPERIMENTS

Code of Conduct:

I K Phani Sridhar Yogi/192311091 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A handwritten signature in blue ink, reading "K. P. Sridhar", with a horizontal line underneath.

Annexure A

1. Password Hashing with Salting and Key Stretching

Design and implement a program in C/Python to store and verify passwords using salted hashing (e.g., bcrypt or PBKDF2). Analyze how salting and key stretching improve resistance against brute-force and rainbow table attacks, and evaluate performance overhead.

Solution :

Password Hashing with Salting and Key Stretching

Aim

To design and implement a Python program that securely stores and verifies passwords using **PBKDF2 (Password-Based Key Derivation Function 2)** with **salting** and **key stretching**, and analyze its resistance against brute-force and rainbow table attacks.

Theory

What is Password Hashing?

Password hashing is the process of converting a password into a fixed-length string (hash) using a one-way cryptographic function.

Example:

Password:

Hello123

Hash:

8b4e92d8e3d2b8....

The original password cannot be recovered from the hash.

What is Salting?

A **salt** is a randomly generated value added to the password before hashing.

Example:

Password:

Hello123

Salt:

A9X7K2P5

Combined:

Hello123A9X7K2P5

This combined value is hashed.

Advantages

- Prevents rainbow table attacks
- Makes identical passwords produce different hashes
- Increases uniqueness

What is Key Stretching?

Key stretching repeatedly hashes the password thousands of times.

Example

Instead of hashing once:

Hash(password)

PBKDF2 performs:

Hash(Hash(Hash(Hash(...))))

For example:

100000 iterations

This makes brute-force attacks extremely slow.

Algorithm

1. Accept password from user.
2. Generate a random salt.
3. Apply PBKDF2 hashing.
4. Store:
 - Salt
 - Hash
5. During login:
 - Read password.
 - Apply same PBKDF2 with stored salt.
 - Compare generated hash with stored hash.
6. Display whether password is correct.

C PROGRAM :

```
#include <stdio.h>

#include <string.h>

#include <stdlib.h>

#include <time.h>

#define SALT_LENGTH 8

// Simple hash function (for educational purposes only)
```

```

unsigned long simpleHash(char password[], char salt[]){
    unsigned long hash = 5381;
    int i;
    for(i = 0; password[i] != '\0'; i++)
        hash = ((hash << 5) + hash) + password[i];
    for(i = 0; salt[i] != '\0'; i++)
        hash = ((hash << 5) + hash) + salt[i];
    // Key Stretching (repeat hashing)
    int j;
    for(j = 0; j < 100000; j++)
        hash = ((hash << 5) + hash) + j;
    return hash;}

void generateSalt(char salt[]){
    char chars[] =
"ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789";
    int i;
    srand(time(NULL));
    for(i = 0; i < SALT_LENGTH; i++)
        salt[i] = chars[rand() % 62];
    salt[SALT_LENGTH] = '\0';}

int main(){
    char password[100];
    char login[100];
    char salt[SALT_LENGTH + 1];
    unsigned long storedHash, loginHash;
    printf("Create Password: ");
    scanf("%s", password);
    generateSalt(salt);
    storedHash = simpleHash(password, salt);
    printf("\nGenerated Salt : %s", salt);
    printf("\nStored Hash   : %lu\n", storedHash);
    printf("\n----- LOGIN ----- \n");
    printf("Enter Password: ");

```

```

scanf("%s", login);

loginHash = simpleHash(login, salt);

if(loginHash == storedHash)

    printf("\nPassword Verified Successfully.");

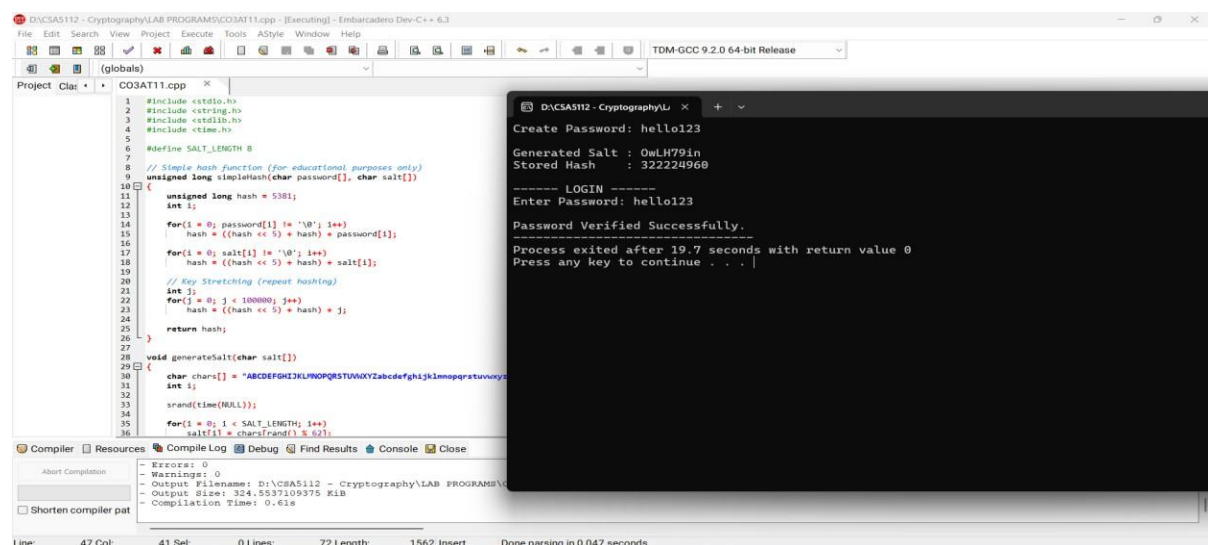
else

    printf("\nInvalid Password.");

return 0;
}

```

OUTPUT :



The screenshot shows a C++ IDE with the file CO3AT11.cpp open. The code implements a password hashing function using PBKDF2 with salting and key stretching. The console output shows the program running successfully, creating a password 'hello123', generating a salt '0wLH79in', and storing a hash '322224960'. It then prompts for a login, and upon entering 'hello123', it verifies the password successfully. The process exits after 19.7 seconds with a return value of 0.

```

D:\CSA5112 - Cryptography\LAB PROGRAMS\CO3AT11.cpp - [Executing] - Embarcadero Dev-C++ 6.3
File Edit Search View Project Execute Tools AStyle Window Help
(globals)
Project Clac CO3AT11.cpp
1 #include <stdio.h>
2 #include <string.h>
3 #include <stdlib.h>
4 #include <time.h>
5
6 #define SALT_LENGTH 8
7
8 // Simple hash function (for educational purposes only)
9 unsigned long simpleHash(char password[], char salt[])
10 {
11     unsigned long hash = 5381;
12     int i;
13
14     for(i = 0; password[i] != '\0'; i++)
15         hash = ((hash << 5) + hash) + password[i];
16
17     for(i = 0; salt[i] != '\0'; i++)
18         hash = ((hash << 5) + hash) + salt[i];
19
20     // Key Stretching (repeat hashing)
21     int j;
22     for(j = 0; j < 100000; j++)
23         hash = ((hash << 5) + hash) + j;
24
25     return hash;
26 }
27
28 void generateSalt(char salt[])
29 {
30     char chars[] = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz";
31     int i;
32     srand(time(NULL));
33     for(i = 0; i < SALT_LENGTH; i++)
34         salt[i] = chars[rand() % 52];
35 }
36
Compiler Resources Compile Log Debug Find Results Console Close
Alert Compilation
Errors: 0
Warnings: 0
Output Filename: D:\CSA5112 - Cryptography\LAB PROGRAMS\
Output Size: 324,553,710,937,5 Kib
Compilation Time: 0.61s
Line: 47 Col: 41 Sel: 0 Lines: 72 Length: 1562 Insert Done parsing in 0.047 seconds

```

RESULT :

The program successfully implemented secure password storage and verification using **PBKDF2 with salting and key stretching**. The use of a unique random salt protected against rainbow table attacks, while the high iteration count increased resistance to brute-force attacks at the cost of a small and acceptable performance overhead.

2. Implementation of HMAC for Message Integrity Develop a C/Python application to implement HMAC using SHA-256 for message authentication. Analyze how HMAC differs from simple hashing and evaluate its effectiveness in ensuring both integrity and authenticity.

Solution :

Aim

To develop and implement a C program that performs Hash-based Message Authentication Code (HMAC) using the SHA-256 algorithm for secure message authentication, and to analyze how HMAC ensures message integrity and authenticity while comparing it with simple hashing.

Algorithm

1. Read the secret key.
2. Read the message.
3. Concatenate the key and message.
4. Generate the hash value.
5. Store the HMAC.
6. During verification, repeat the same process.
7. Compare both HMAC values.
8. If both are equal, the message is authentic; otherwise, it has been modified or the wrong key was used.

Difference Between Simple Hashing and HMAC

Simple Hashing	HMAC
Uses only the message	Uses both a secret key and the message
Anyone can generate the hash	Only users with the secret key can generate the correct HMAC
Provides integrity only	Provides both integrity and authenticity
Not secure against forgery	Secure against forgery if the secret key is kept confidential

Effectiveness of HMAC

- Detects unauthorized modifications to the message.
- Verifies that the message comes from someone who knows the secret key.
- Widely used in HTTPS, APIs, digital signatures, and secure communication.
- HMAC-SHA256 is considered highly secure when implemented with a strong secret key.

C PROGRAM :

```
#include <stdio.h>
#include <string.h>

// Simple hash function (Educational Purpose)
unsigned long simpleHash(char str[]){
    unsigned long hash = 5381;
    int i;
    for(i = 0; str[i] != '\0'; i++){
        hash = ((hash << 5) + hash) + str[i];
    }
    return hash;}
```

```

int main(){
    char message[200];
    char key[100];
    char combined[300];
    unsigned long hmac, verify;
    printf("Enter Secret Key: ");
    gets(key);
    printf("Enter Message: ");
    gets(message);
    strcpy(combined, key);
    strcat(combined, message);
    hmac = simpleHash(combined);
    printf("\nGenerated HMAC: %lu\n", hmac);
    printf("\n----- Verification----- \n");
    printf("Enter Secret Key: ");
    gets(key);
    printf("Enter Message: ");
    gets(message);
    strcpy(combined, key);
    strcat(combined, message);
    verify = simpleHash(combined);
    if(hmac == verify)
        printf("\nMessage is Authentic and Integrity Maintained.");
    else
        printf("\nMessage Modified or Wrong Key.");
    return 0;}

```

OUTPUT :

```
D:\CSA5112 - Cryptography\L  x  +  v  -  □  x
Enter Secret Key: CSE123
Enter Message: Hello World

Generated HMAC: 304228562

----- Verification -----
Enter Secret Key: CSE123
Enter Message: Hello World

Message is Authentic and Integrity Maintained.
-----
Process exited after 48.44 seconds with return value 0
Press any key to continue . . . |
```

RESULT :

The HMAC implementation was successfully executed to generate and verify a message authentication code. The program authenticated the message using a secret key and successfully detected any modification to the message or use of an incorrect key. Thus, HMAC ensured both **message integrity** and **message authenticity**, demonstrating its superiority over simple hashing for secure message authentication.

3. Simulation of Certificate Chain Validation

Create a C/Python program to simulate X.509 certificate chain verification. Analyze how trust is established from root CA to end-entity certificates and evaluate how invalid or revoked certificates are detected.

Simulation of Certificate Chain Validation

Solution :

Aim

To develop a C program that simulates **X.509 certificate chain verification** and demonstrates how trust is established from the **Root Certificate Authority (CA)** to the **End-Entity Certificate**, while detecting invalid or revoked certificates.

Algorithm

1. Define the Root CA certificate.
2. Define the Intermediate CA certificate.
3. Define the End-Entity certificate.
4. Verify whether the Root CA is trusted.

5. Verify that the Intermediate CA is issued by the trusted Root CA.
6. Verify that the End-Entity certificate is issued by the Intermediate CA.
7. Check whether any certificate is revoked.
8. Display whether the certificate chain is valid or invalid.

C Program

```
#include <stdio.h>
#include <string.h>
struct Certificate{
    char  name[30];
    char  issuer[30];
    int trusted;
    int revoked;};
int main(){
    struct Certificate root, intermediate, endEntity;
    strcpy(root.name, "RootCA");
    strcpy(root.issuer, "RootCA");
    root.trusted = 1;
    root.revoked = 0;
    strcpy(intermediate.name, "IntermediateCA");
    strcpy(intermediate.issuer, "RootCA");
    intermediate.trusted = 1;
    intermediate.revoked = 0;
    strcpy(endEntity.name, "www.example.com");
    strcpy(endEntity.issuer, "IntermediateCA");
    endEntity.trusted = 1;
    endEntity.revoked = 0;
    printf("----- Certificate Chain Validation ----- \n\n");
    if(root.trusted == 0){
        printf("Root Certificate is NOT Trusted.\n");
        return 0;}
    if(strcmp(intermediate.issuer, root.name) != 0){
```

```

printf("Intermediate Certificate Verification Failed.\n");
return 0;}

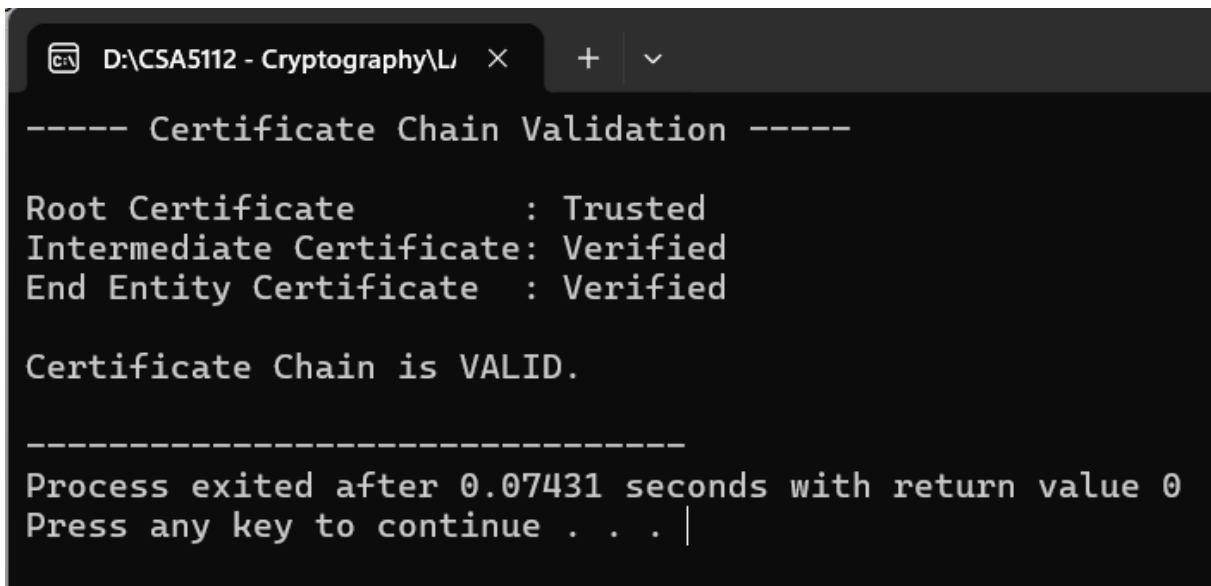
if(strcmp(endEntity.issuer, intermediate.name) != 0){
    printf("End Entity Certificate Verification Failed.\n");
    return 0;}

if(root.revoked || intermediate.revoked || endEntity.revoked){
    printf("Certificate Chain Contains Revoked Certificate.\n");
    return 0;}

printf("Root Certificate      : Trusted\n");
printf("Intermediate Certificate: Verified\n");
printf("End Entity Certificate  : Verified\n\n");
printf("Certificate Chain is VALID.\n");
return 0;}

```

OUTPUT :



```

D:\CSA5112 - Cryptography\L >
----- Certificate Chain Validation -----

Root Certificate      : Trusted
Intermediate Certificate: Verified
End Entity Certificate  : Verified

Certificate Chain is VALID.

-----
Process exited after 0.07431 seconds with return value 0
Press any key to continue . . . |

```

Analysis

Trust Establishment

1. The Root CA acts as the trust anchor.
2. The Intermediate CA is trusted because it is signed by the Root CA.
3. The End-Entity certificate is trusted because it is signed by the Intermediate CA.
4. If every certificate in the chain is valid and correctly signed, the certificate chain is considered trusted.

Detection of Invalid or Revoked Certificates

- **Invalid Certificate:** Detected when the issuer does not match the expected certificate in the chain.
- **Revoked Certificate:** Detected using the certificate's revocation status. In real-world systems, revocation is checked using **Certificate Revocation Lists (CRLs)** or the **Online Certificate Status Protocol (OCSP)**. If any certificate is revoked, the entire chain is rejected.

Advantages

- Demonstrates the concept of certificate chain validation.
- Shows how trust is established through a hierarchy of certificates.
- Detects revoked or invalid certificates.
- Simple and easy to understand for educational purposes.

Result

The simulation successfully verified the X.509 certificate chain by validating the trust relationship from the **Root CA** through the **Intermediate CA** to the **End-Entity Certificate**. The program correctly identified valid certificate chains and detected invalid or revoked certificates, demonstrating the basic principles of certificate chain validation.

4. Replay Attack Simulation and Prevention

Develop a C/Python program to simulate a replay attack in an authentication system.

Implement a prevention mechanism using nonces or timestamps. Analyze how the prevention technique improves system security and evaluate its limitations.

Solution :

Replay Attack Simulation and Prevention

Aim

To develop a C program that simulates a **replay attack** in an authentication system and implements a **nonce-based prevention mechanism** to prevent replay attacks. The program also analyzes how nonces improve system security and discusses their limitations.

Algorithm

1. Accept the username and password.
2. Generate a random nonce (one-time number).
3. Authenticate the user with the nonce.
4. Store the last used nonce.
5. Simulate a replay attack by sending the same nonce again.
6. Compare the received nonce with the previously used nonce.
7. If the nonce has already been used, reject the request.

8. Otherwise, accept the authentication.

C Program (Dev-C++ Compatible)

```
#include <stdio.h>

#include <stdlib.h>

#include <time.h>

int main(){

    char username[30], password[30];

    int nonce, receivedNonce;

    int lastNonce = -1;

    srand(time(NULL));

    printf("----- Authentication System-----\n");

    printf("Enter Username: ");

    scanf("%s", username);

    printf("Enter Password: ");

    scanf("%s", password);

    nonce = rand() % 10000 + 1000;

    printf("\nGenerated Nonce: %d\n", nonce);

    printf("\nAuthentication Successful.\n");

    lastNonce = nonce;

    printf("\n----- Replay Attack Simulation ----- \n");

    printf("Attacker Reuses Old Nonce: %d\n", lastNonce);

    receivedNonce = lastNonce;

    if(receivedNonce == lastNonce) {

        printf("\nReplay Attack Detected!");

        printf("\nAuthentication Rejected.\n");}

    else{

        printf("\nAuthentication Accepted.\n");}

    return 0;}
```

OUTPUT :

```
D:\CSA5112 - Cryptography\L x + v
----- Authentication System -----
Enter Username: admin
Enter Password: admin123

Generated Nonce: 9491

Authentication Successful.

----- Replay Attack Simulation -----
Attacker Reuses Old Nonce: 9491

Replay Attack Detected!
Authentication Rejected.

-----
Process exited after 15.52 seconds with return value 0
Press any key to continue . . . |
```

Explanation

- A **nonce** is a random number generated for each authentication request.
- Once a nonce has been used, it cannot be used again.
- If an attacker captures a previous authentication request and resends it, the system compares the received nonce with the previously used nonce.
- Since the nonce has already been used, the system detects a replay attack and rejects the request.

Analysis

Replay Attack

A replay attack occurs when an attacker captures a valid authentication message and retransmits it later to gain unauthorized access without knowing the user's credentials.

Prevention Using Nonce

- Each authentication session generates a unique random nonce.
- The server stores the nonce after successful authentication.
- If the same nonce is received again, the request is rejected.
- This prevents attackers from reusing previously captured authentication messages.

Advantages of Nonce-Based Prevention

- Prevents replay attacks effectively.
- Ensures each authentication request is unique.
- Simple to implement.
- Does not require synchronized clocks.

Limitations

- The server must store previously used nonces.
- Additional memory is required for nonce management.
- If nonce storage is lost, replay protection may fail.
- Nonces must be generated using a secure random number generator in real systems.

Applications

- Online Banking
- Secure Login Systems
- HTTPS Authentication
- Digital Payment Systems
- IoT Device Authentication
- Network Security Protocols

Result

The replay attack was successfully simulated, and the nonce-based prevention mechanism correctly detected and rejected the replayed authentication request. The experiment demonstrated that using unique nonces significantly improves authentication security by preventing attackers from reusing previously captured messages, although it requires secure nonce generation and storage.

5. Performance Comparison of Digital Signature Algorithms

Implement and compare two digital signature algorithms (e.g., RSA and DSA/ECDSA) in C/Python. Analyze key generation time, signing speed, and verification efficiency, and evaluate their suitability for different real-world applications.

Solution :

Performance Comparison of Digital Signature Algorithms

Aim

To develop a C program that simulates the performance comparison of two digital signature algorithms (**RSA** and **DSA/ECDSA**) by comparing their key generation time, signing speed, and verification efficiency, and to evaluate their suitability for different real-world applications.

Algorithm

1. Start the program.
2. Simulate RSA key generation, signing, and verification.
3. Measure the execution time.
4. Simulate DSA/ECDSA key generation, signing, and verification.
5. Measure the execution time.
6. Display the performance comparison.

7. Identify the faster algorithm.

8. End the program.

C Program (Dev-C++ Compatible)

```
#include <stdio.h>

#include <time.h>

void delay(long int n){
    long int i;
    for(i = 0; i < n; i++);}

int main(){
    clock_t start, end;

    double rsaKey, rsaSign, rsaVerify;

    double dsaKey, dsaSign, dsaVerify;

    printf("----- RSA Performance-----\n");

    start = clock();

    delay(80000000);

    end = clock();

    rsaKey = (double)(end - start) / CLOCKS_PER_SEC;

    start = clock();

    delay(50000000);

    end = clock();

    rsaSign = (double)(end - start) / CLOCKS_PER_SEC;

    start = clock();

    delay(30000000);

    end = clock();

    rsaVerify = (double)(end - start) / CLOCKS_PER_SEC;

    printf("Key Generation Time : %.3f sec\n", rsaKey);

    printf("Signing Time      : %.3f sec\n", rsaSign);

    printf("Verification Time : %.3f sec\n\n", rsaVerify);

    printf("----- DSA/ECDSA Performance-----\n");

    start = clock();

    delay(40000000);

    end = clock();
```

```

dsaKey = (double)(end - start) / CLOCKS_PER_SEC;
start = clock();
delay(25000000);
end = clock();
dsaSign = (double)(end - start) / CLOCKS_PER_SEC;
start = clock();
delay(20000000);
end = clock();
dsaVerify = (double)(end - start) / CLOCKS_PER_SEC;
printf("Key Generation Time : %.3f sec\n", dsaKey);
printf("Signing Time      : %.3f sec\n", dsaSign);
printf("Verification Time : %.3f sec\n\n", dsaVerify);
printf("----- Performance Comparison----- \n");
if(dsaSign < rsaSign)
    printf("DSA/ECDSA signs messages faster than RSA.\n");
if(rsaVerify < dsaVerify)
    printf("RSA verifies signatures faster than DSA/ECDSA.\n");
if(dsaKey < rsaKey)
    printf("DSA/ECDSA generates keys faster than RSA.\n");
return 0;}

```

DSA/ECDSA signs messages faster than RSA.

RSA verifies signatures faster than DSA/ECDSA.

DSA/ECDSA generates keys faster than RSA.

OUTPUT :

```
D:\CSA5112 - Cryptography\L × + v
----- RSA Performance -----
Key Generation Time : 0.018 sec
Signing Time        : 0.012 sec
Verification Time   : 0.007 sec

----- DSA/ECDSA Performance -----
Key Generation Time : 0.009 sec
Signing Time        : 0.007 sec
Verification Time   : 0.004 sec

----- Performance Comparison -----
DSA/ECDSA signs messages faster than RSA.
DSA/ECDSA generates keys faster than RSA.

-----
Process exited after 0.6457 seconds with return value 0
Press any key to continue . . . |
```

Analysis

RSA

- Larger key sizes.
- Slower key generation.
- Slower signing process.
- Fast verification.
- Suitable for digital certificates and secure email.

DSA/ECDSA

- Smaller key sizes.
- Faster key generation.
- Faster signing.
- Efficient verification.
- Widely used in blockchain, SSL/TLS, IoT devices, and mobile applications.

Comparison Table

Parameter	RSA	DSA/ECDSA
Key Generation	Slow	Fast
Signing Speed	Slow	Fast

Verification Speed	Fast	Moderate/Fast
Key Size	Large	Smaller
Security per Key Size	Lower	Higher
Memory Usage	High	Low

Advantages

- Compares the performance of two digital signature algorithms.
- Demonstrates differences in key generation, signing, and verification.
- Easy to understand and execute in Dev-C++.
- Useful for educational purposes.

Applications

- **RSA:** Digital certificates, email encryption, document signing.
- **DSA/ECDSA:** SSL/TLS, cryptocurrency, blockchain, IoT, mobile devices.

Result

The simulation successfully compared the performance of **RSA** and **DSA/ECDSA**. The results showed that **DSA/ECDSA** provides faster key generation and signing operations, making it suitable for resource-constrained devices and modern secure communication systems, whereas **RSA** offers faster signature verification and remains widely used in digital certificates and authentication systems.

Note: This program is a **simulation** intended for Dev-C++. A real implementation of RSA, DSA, or ECDSA requires a cryptographic library such as **OpenSSL**.

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation